Initial Design Submission

Advanced Web Development

Submitted on: 20.01.2026

Submitted By:

Pawan Lamsal
1751362
lamsal@uni-hildesheim.de

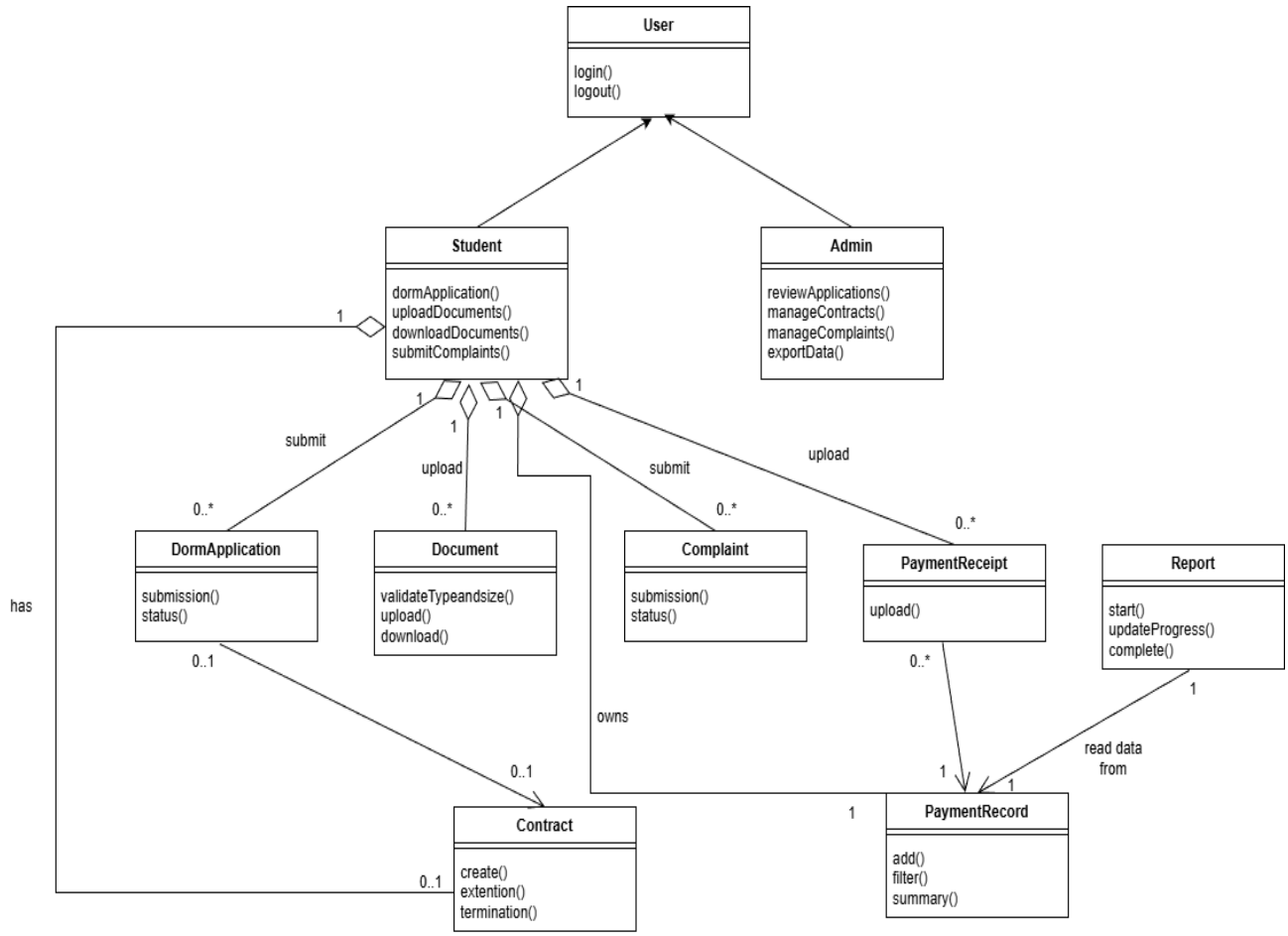
Sher Afgan
1751148
afgan@uni-hildesheim.de

Suman Neupane
1751373
neupane@uni-hildesheim.de

Student Dorm Management System

The Student Dormitory Management System, a web-based application intended to support and digitalize core administrative processes in student dormitories. The system targets two user roles, students and dormitory administrators, and defines functionality for online dormitory applications, tenant profile management, digital contract handling, rent payment record management, and complaint submission and processing. The requirements include mechanisms for uploading and downloading documents such as contracts and rent payment receipts, as well as a backend process for generating rent payment history reports as a longer-running computation with progress feedback displayed on the frontend. In addition, the system provides role-based access control, dialog-based user confirmations, and router-based menu navigation.

Structural Overview – Class Diagram



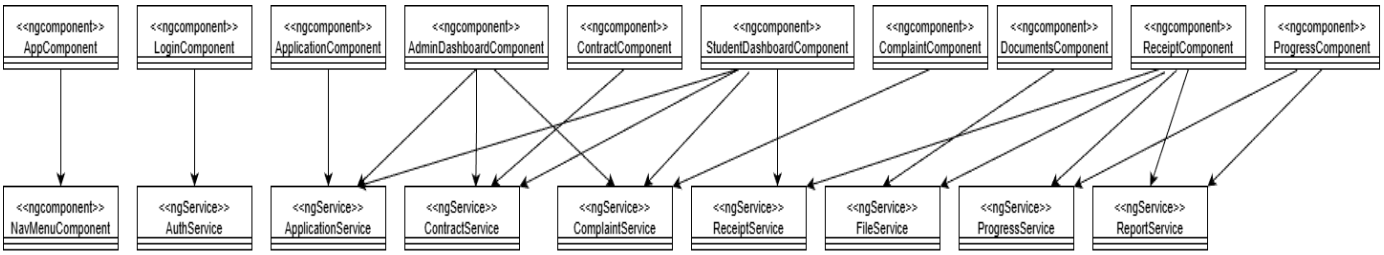
- **Components and Services**

  <<ngComponent>>: ngComponents are responsible for managing the user interface and interactions which are listed as follows:

- App Component
- Nav Menu Component
- Login Component
- Application Component
- Admin Dashboard Component
- Student Dashboard Component
- Contract Component
- Complaint Component
- Documents Component
- Receipt Component
- Progress Component

<<ngService>>: ngServices encapsulate the business logic and handles interactions between the components and database which are listed as follows:

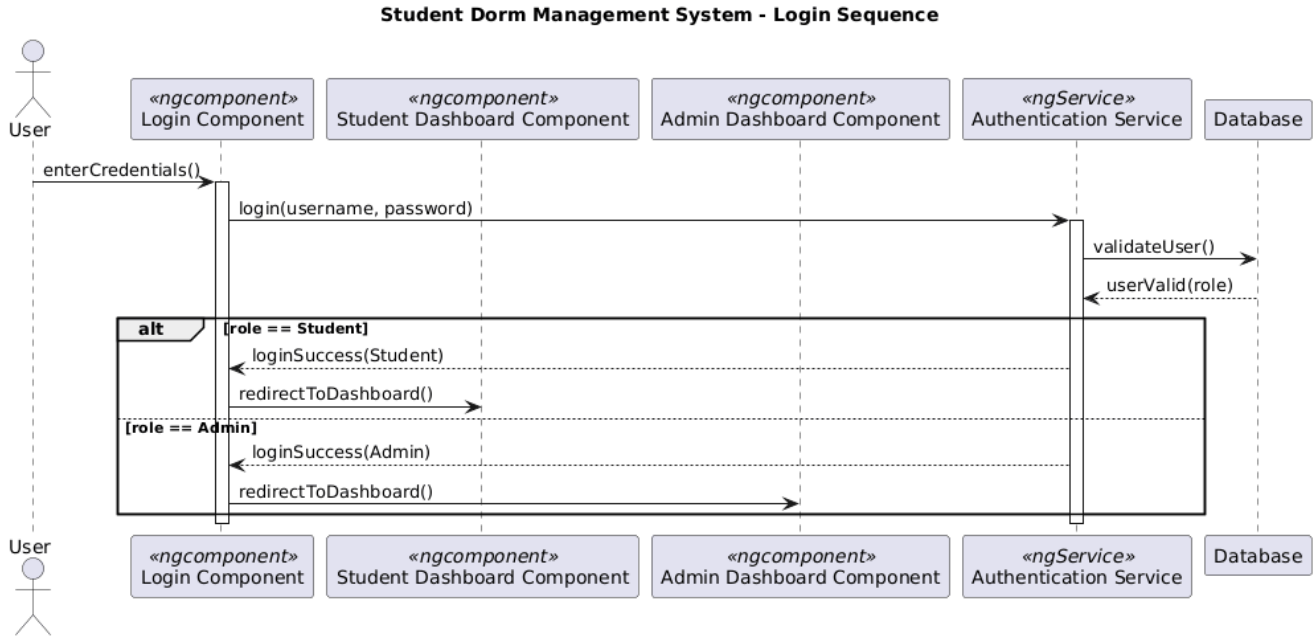
- Authentication Service
- Application Service
- Contract Service
- Complaint Service
- Receipt Service
- File Service
- Report Service
- Progress Service



Behavioral Overview – Sequence Diagrams

Each of the following sequence diagram represents a concrete scenario of a use case and illustrates the interaction between actors, Angular components, and backend services.

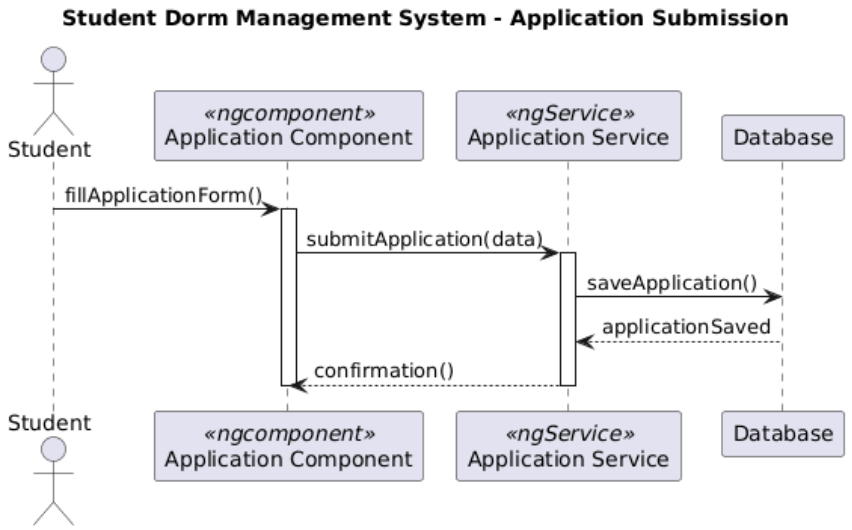
- **Login Scenario**



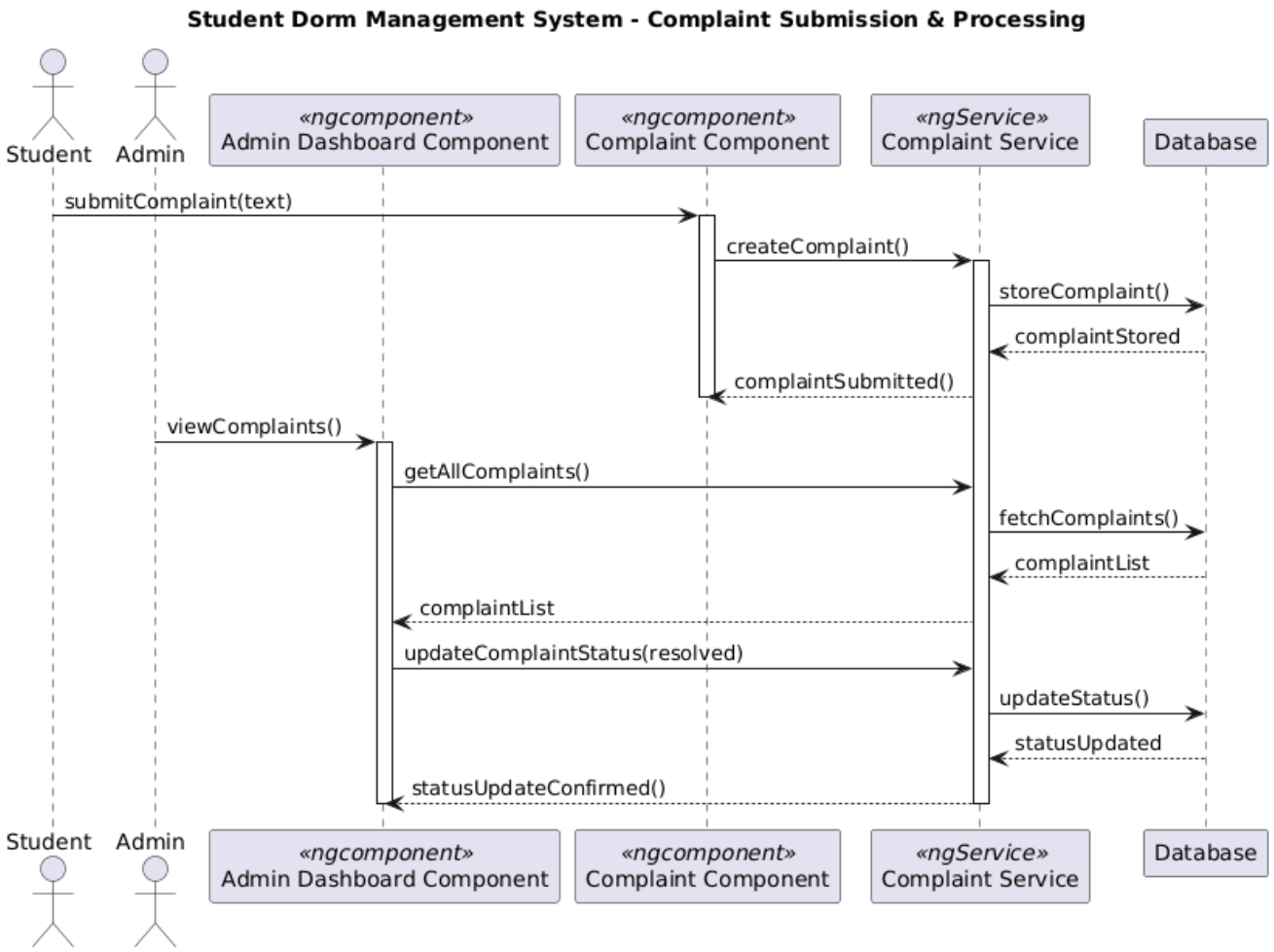
  - The user enters credentials via the LoginComponent.
  - The component forwards the credentials to the AuthenticationService.
  - The service validates the credentials with the Database.
  - Based on the returned role (Student or Administrator), the user is redirected to the corresponding dashboard.

- **Application Submission Scenario**

  - The student fills out and submits the application using the ApplicationComponent**.**
  - The component sends the application data to the ApplicationService**.**
  - The service stores the application in the Database**.**
  - A confirmation is returned and displayed to the student.

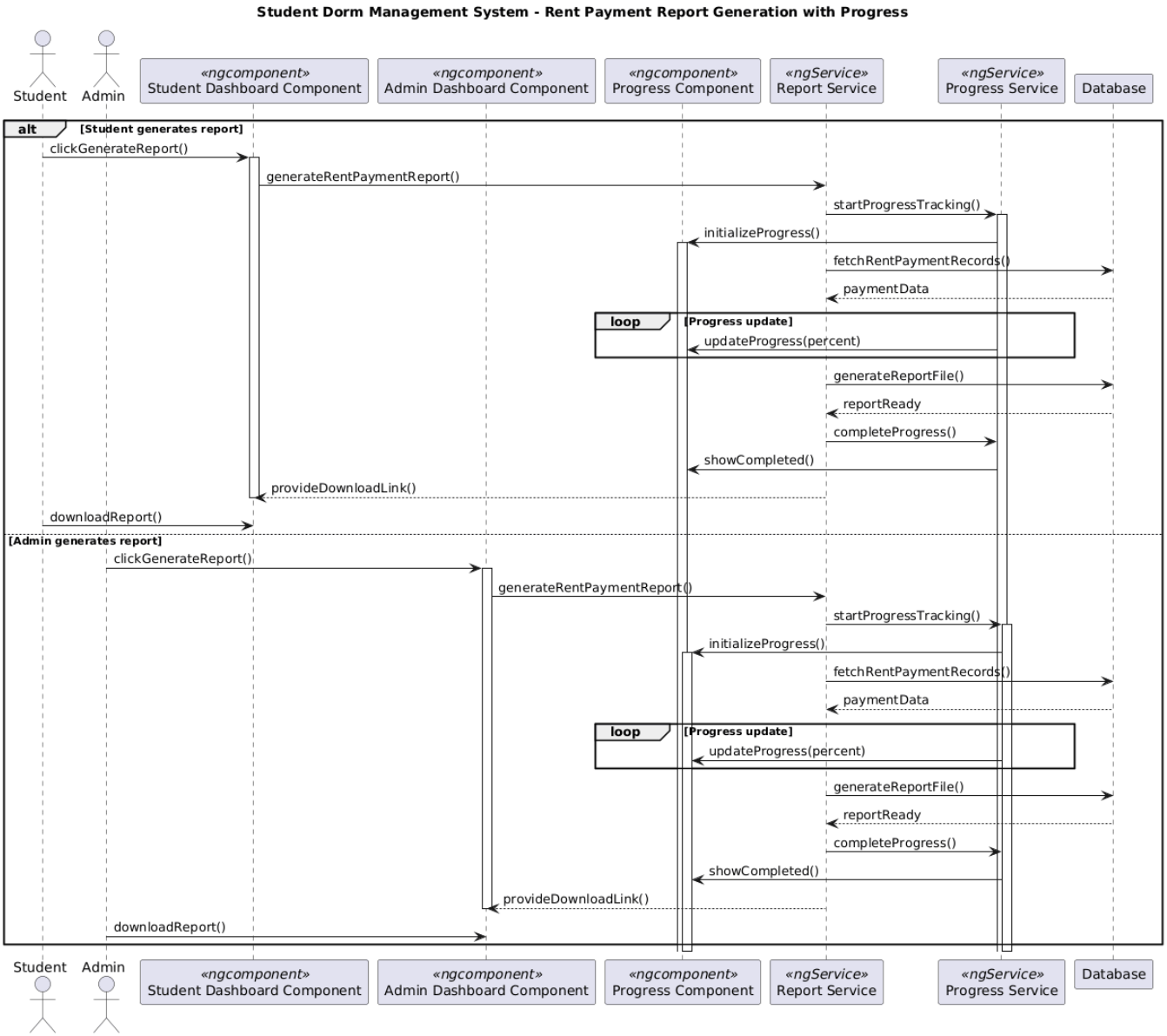


- **Complaint Submission and Processing Scenario**



- The student submits a complaint through the ComplaintComponent.
- The component delegates the request to the ComplaintService, which stores it in the Database.
- Administrators retrieve complaints via the AdminDashboardComponent.
- Complaint status updates are processed by the ComplaintService and confirmed back to the administrator.

- **Rent Payment Report Generation with Progress**



- A student or administrator initiates report generation from their respective dashboard.
- The request is sent to the ReportService, which starts a long-running backend computation.
- The ProgressService tracks execution progress and updates the ProgressComponent.
- Once complete, a download link for the generated report is provided to the initiating user.

- **Actors and Participants**

  - **Student**: Interacts with the system to log in, submit dormitory applications, upload documents, submit complaints, and generate rent payment reports.
  - **Admin**: Uses the system to review applications, manage complaints, generate rent payment reports, and administer dormitory contracts.
  - **<<ngComponent>> LoginComponent**: Manages user authentication and routes users to the appropriate dashboard based on their role.
  - **<<ngComponent>> StudentDashboardComponent**: Provides students with access to application status, complaint submission, document handling, and report generation.
  - **<<ngComponent>> AdminDashboardComponent**: Enables administrators to manage applications, complaints, contracts, and generate reports.
  - **<<ngComponent>> ApplicationComponent**: Handles dormitory application form input and submission.
  - **<<ngComponent>> ComplaintComponent**: Manages complaint submission and status updates.
  - **<<ngComponent>> ProgressComponent**: Displays progress information for long-running backend tasks.
  - **<<ngService>> AuthenticationService**: Handles authentication logic and role validation.
  - **<<ngService>> ApplicationService**: Processes dormitory applications and persists them in the database.
  - **<<ngService>> ComplaintService**: Handles complaint creation, retrieval, and status updates.
  - **<<ngService>> ReportService**: Generates rent payment history reports as a long-running backend task.
  - **<<ngService>> ProgressService**: Tracks and communicates the progress of long-running backend computations.
  - **Database**: Stores user accounts, applications, complaints, contracts, payment records, and generated reports.

- **Interaction Flow**
  - User actions are handled by Angular components, which manage UI interactions.
  - Business logic and data processing are encapsulated in Angular services.
  - Services communicate with the Database to store and retrieve persistent data.
  - Long-running backend operations provide progress feedback to the frontend via the ProgressService and ProgressComponent

Data

This section describes the core data structures of the Student Dorm Management System, including the relationships between entities.

- **Entities and Attributes**
  - **User**
    - **userId**: Unique identifier for the user.
    - **name**: Full name of the user.
    - **email**: User's email address.
    - **password**: Encrypted password for authentication.
    - **role**: Role of the user (Student or Admin).
  - **StudentProfile**
    - **profileId**: Unique identifier for the student profile.
    - **userId**: Reference to the associated user.
    - **academicDetails**: Academic and personal information of the student.
    - **applicationStatus**: Current dormitory application status.
    - **roomInformation**: Assigned room details (if applicable).
  - **DormApplication**
    - **applicationId**: Unique identifier for the application.
    - **studentId**: Reference to the student submitting the application.
    - **submissionDate**: Date of application submission.
    - **status**: Application status (Pending, Accepted, Rejected).
    - **documents**: Uploaded supporting documents.
  - **Contract**
    - **contractId**: Unique identifier for the contract.
    - **studentId**: Reference to the student.
    - **startDate**: Contract start date.
    - **endDate**: Contract end date.
    - **status**: Contract status (Active, Extended, Terminated).
    - **signedDocument**: Uploaded signed contract file.
  - **RentPayment**
    - **paymentId**: Unique identifier for the payment record.
    - **studentId**: Reference to the student.
    - **month**: Payment month.
    - **amount**: Rent amount paid.
    - **receiptFile**: Uploaded payment receipt.
  - **Complaint**
    - **complaintId**: Unique identifier for the complaint.
    - **studentId**: Reference to the student who submitted the complaint.
    - **description**: Complaint details.
    - **status**: Complaint status (Submitted, In Progress, Resolved).
    - **createdAt**: Complaint submission date.
  - **Report**
    - **reportId**: Unique identifier for the generated report.
    - **generatedBy**: User who initiated the report generation.
    - **generationDate**: Date of report creation.

    - **filePath**: Location of the generated report file.
- **Relationships**
    - A **User** has exactly one role (Student or Admin).
    - A **Student** has one **StudentProfile** (1-to-1).
    - A **Student** can submit multiple **DormApplications** (1-to-Many).
    - A **Student** can have one active **Contract** at a time (1-to-1).
    - A **Student** can have multiple **RentPayments** (1-to-Many).
    - A **Student** can submit multiple **Complaints** (1-to-Many).
    - A **User** can generate multiple **Reports** (1-to-Many).

Screen Mocks

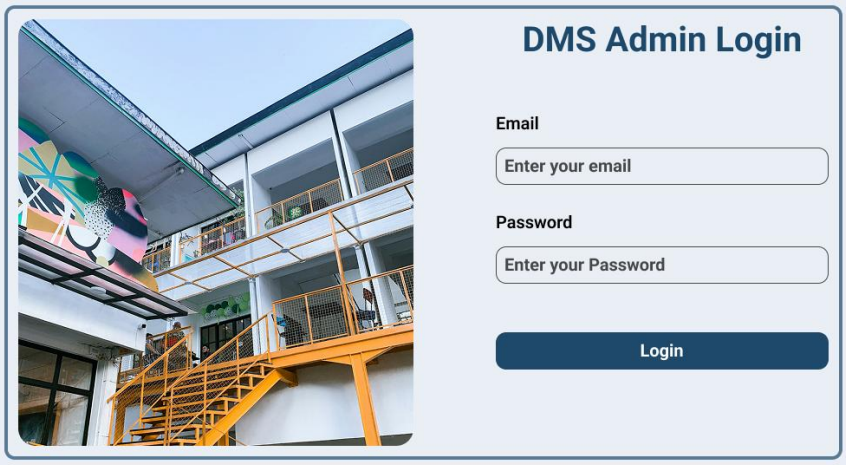


Figure 1: Mock Screen - Admin Login

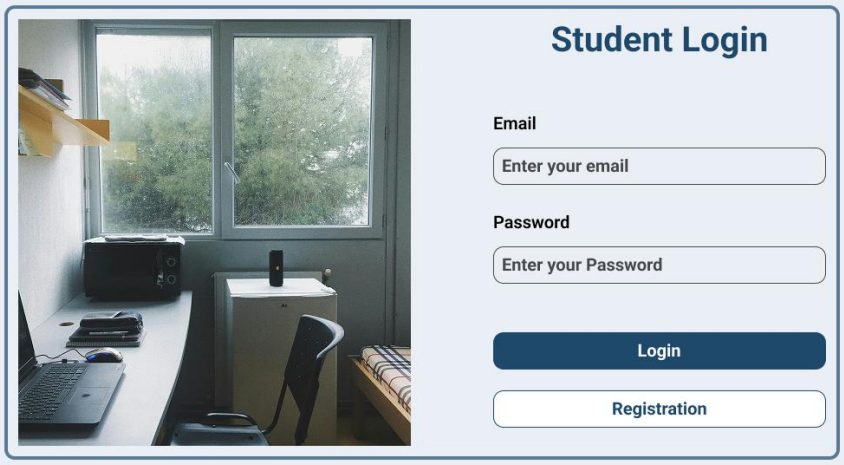


Figure 2: Mock Screen - Student Login

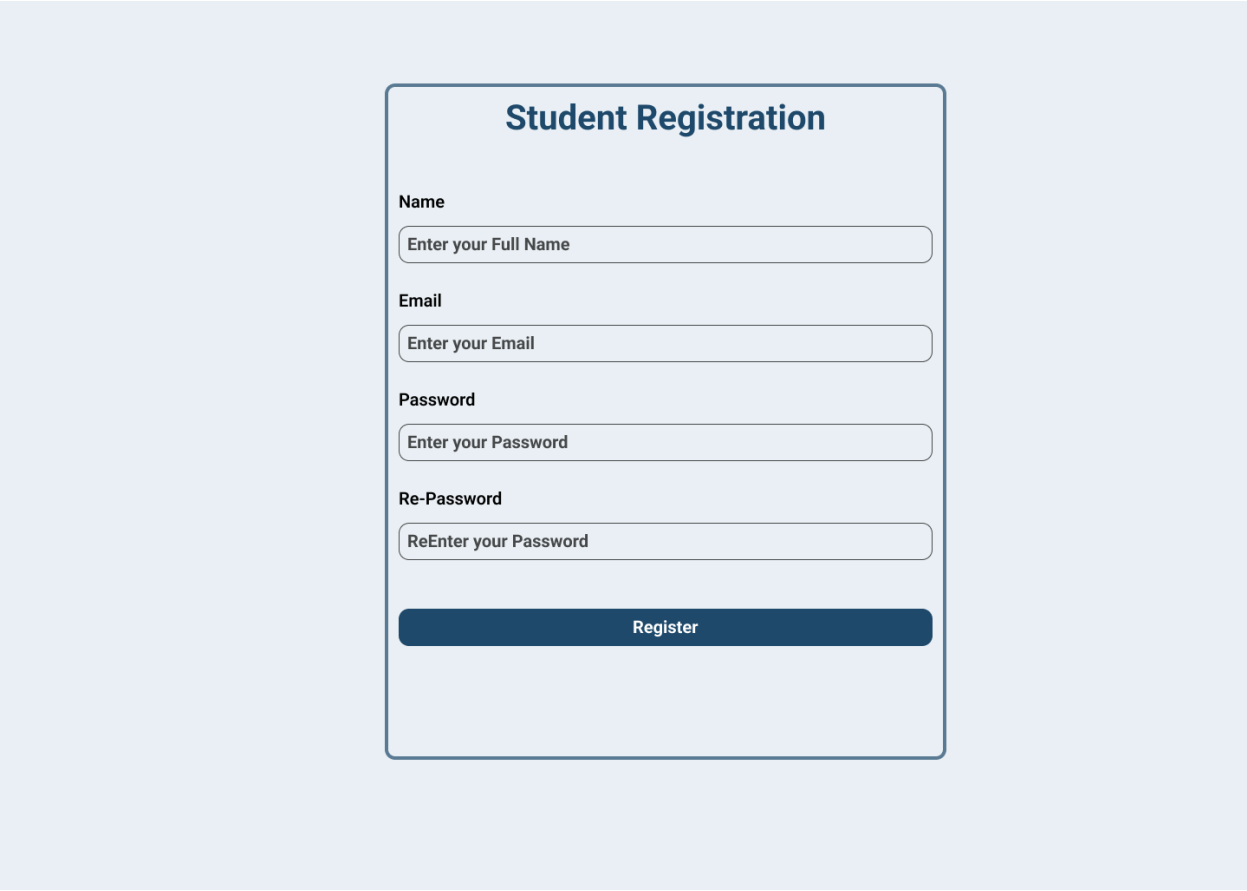


Figure 3: Mock Screen - Student Registration

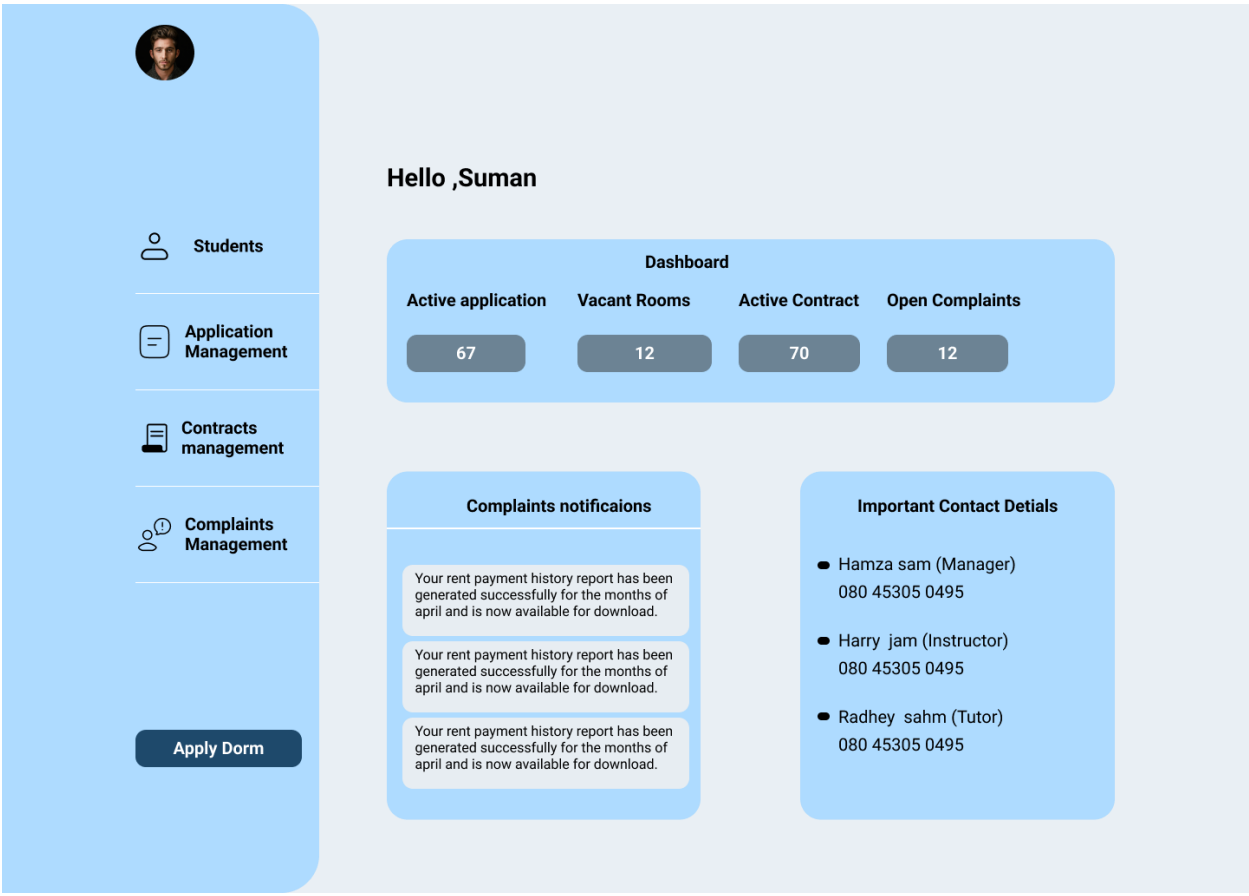


Figure 4: Mock Screen - Admin Dashboard

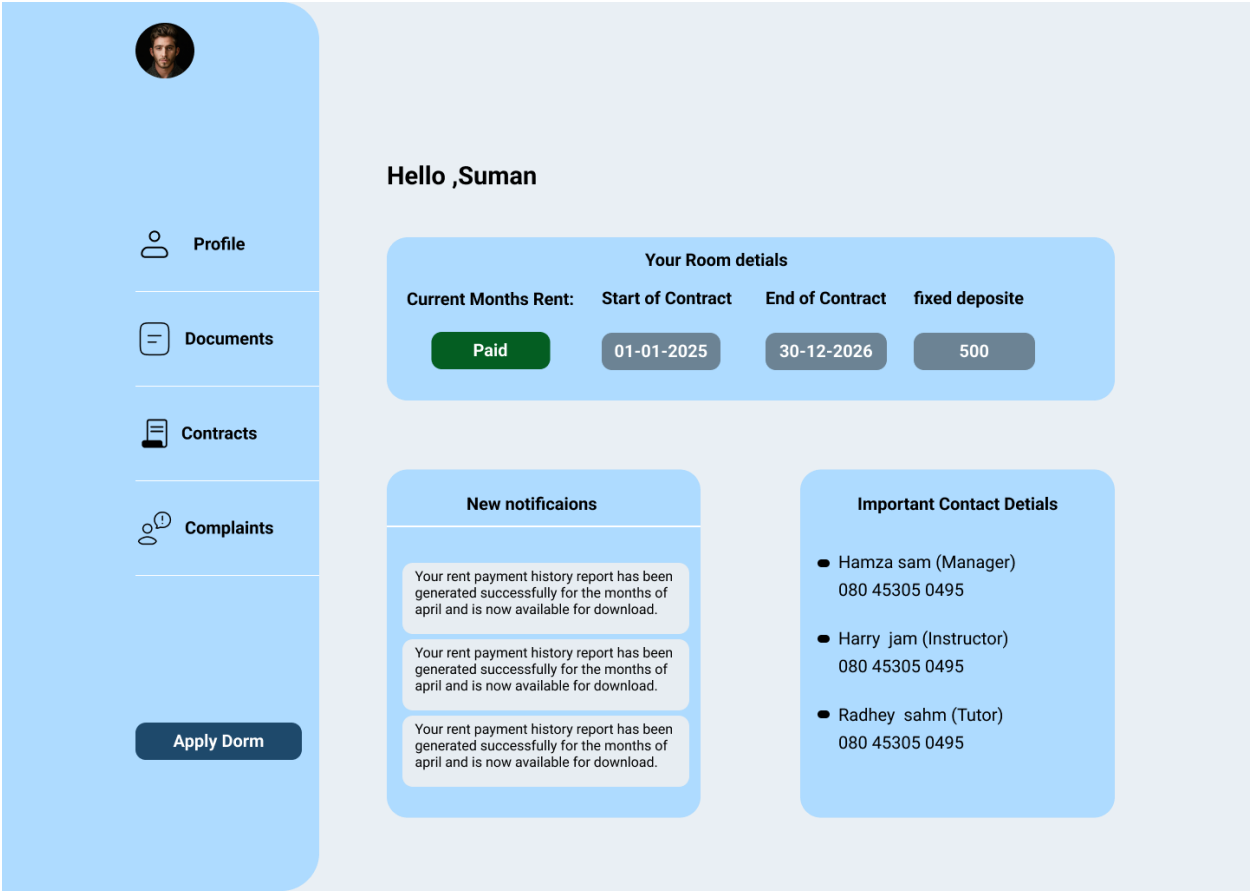


Figure 5: Mock Screen - Student Dashboard

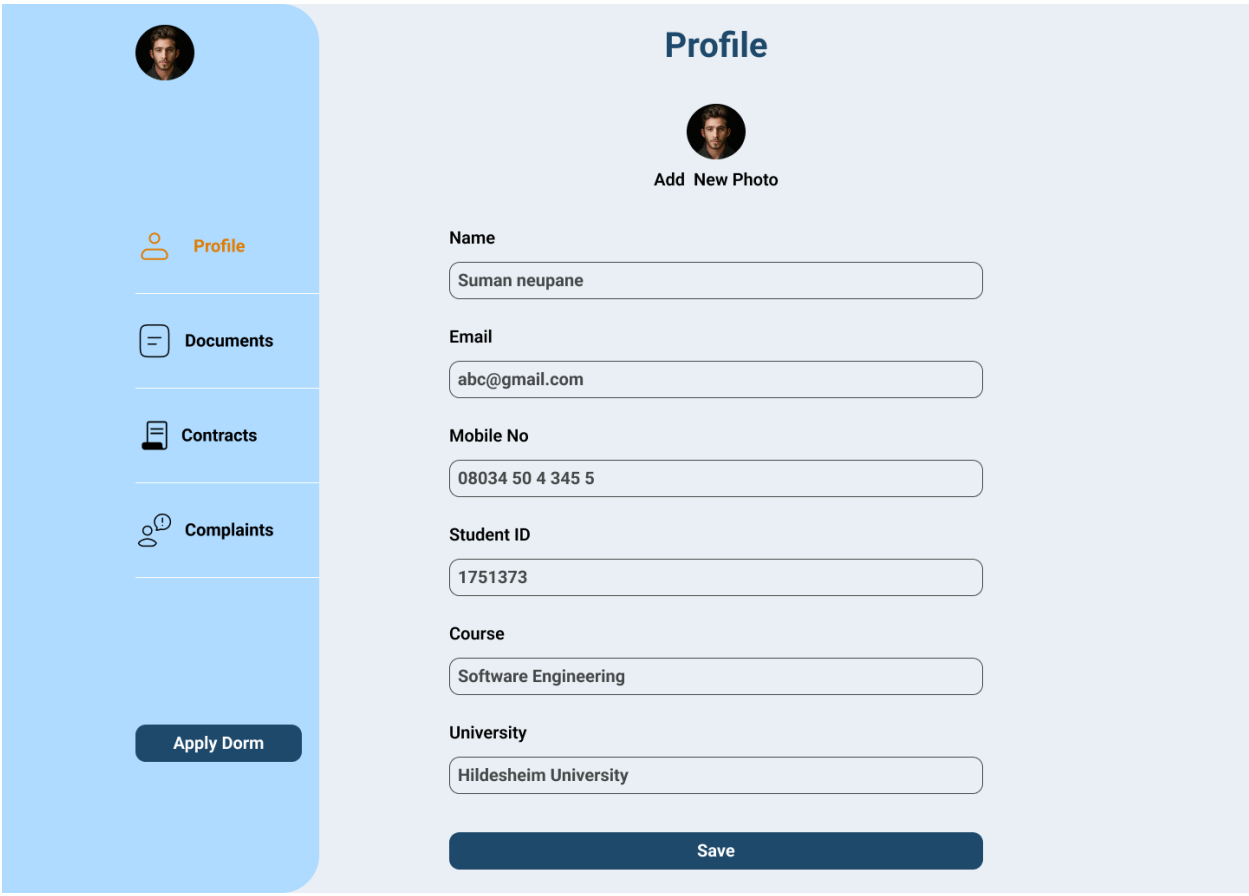


Figure 6: Mock Screen - Student Profile

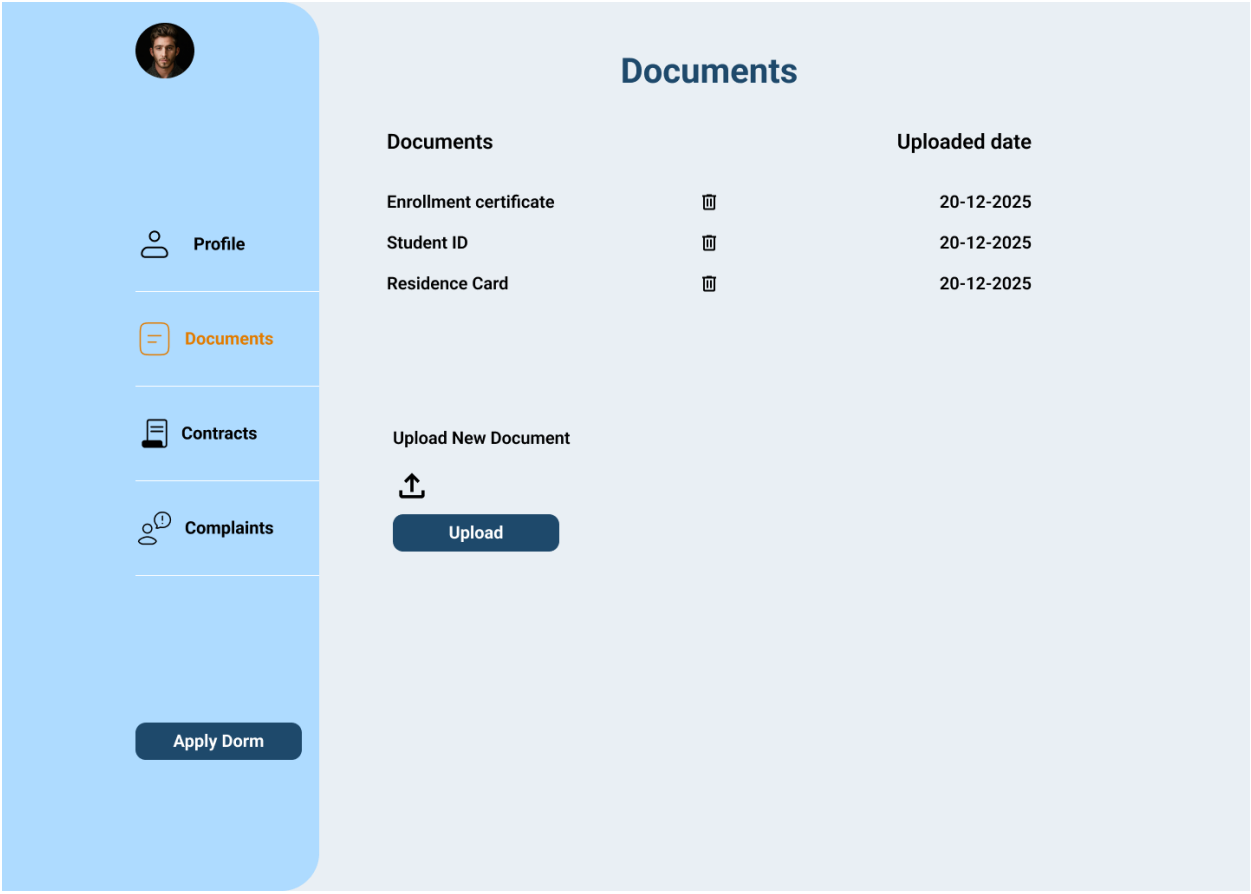


Figure 7: Mock Screen - Student Document

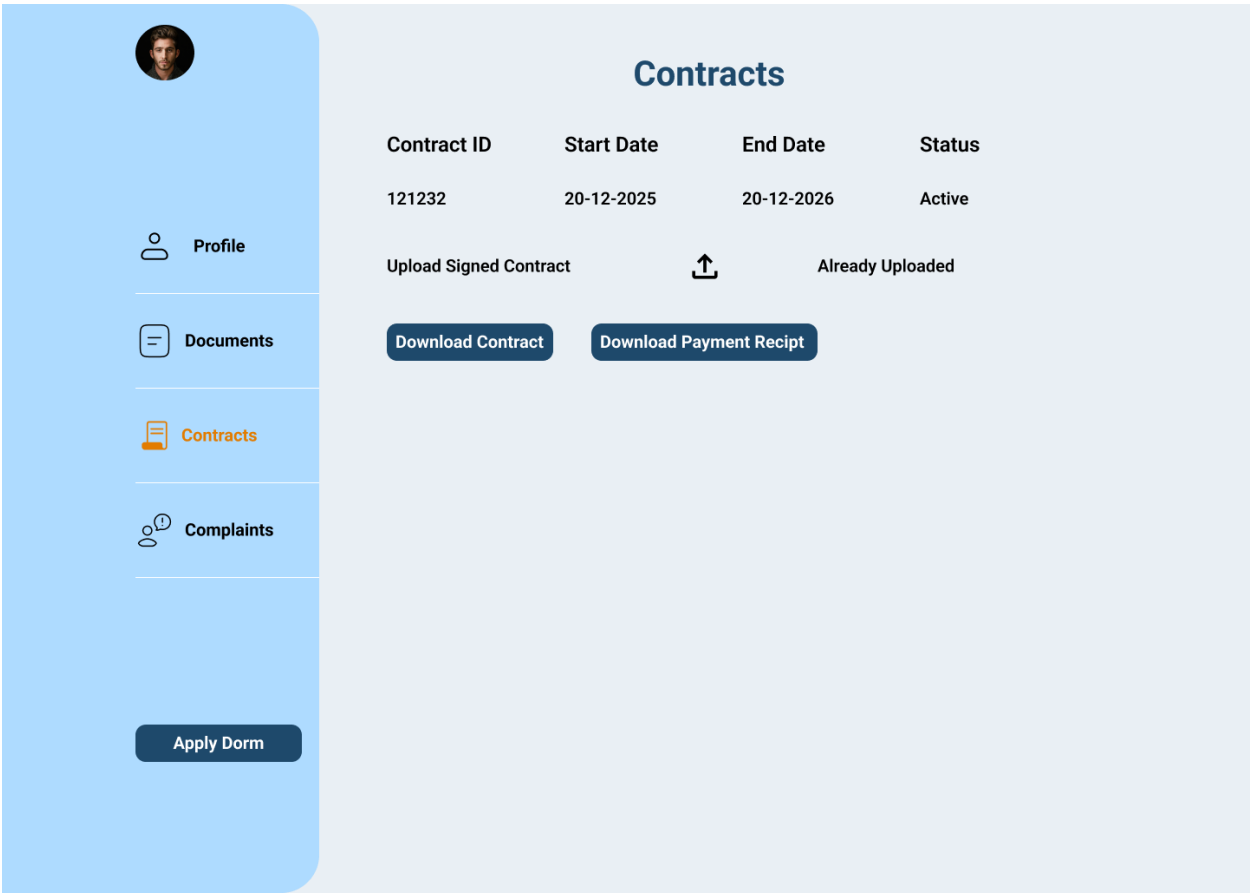


Figure 8: Mock Screen - Student Contracts

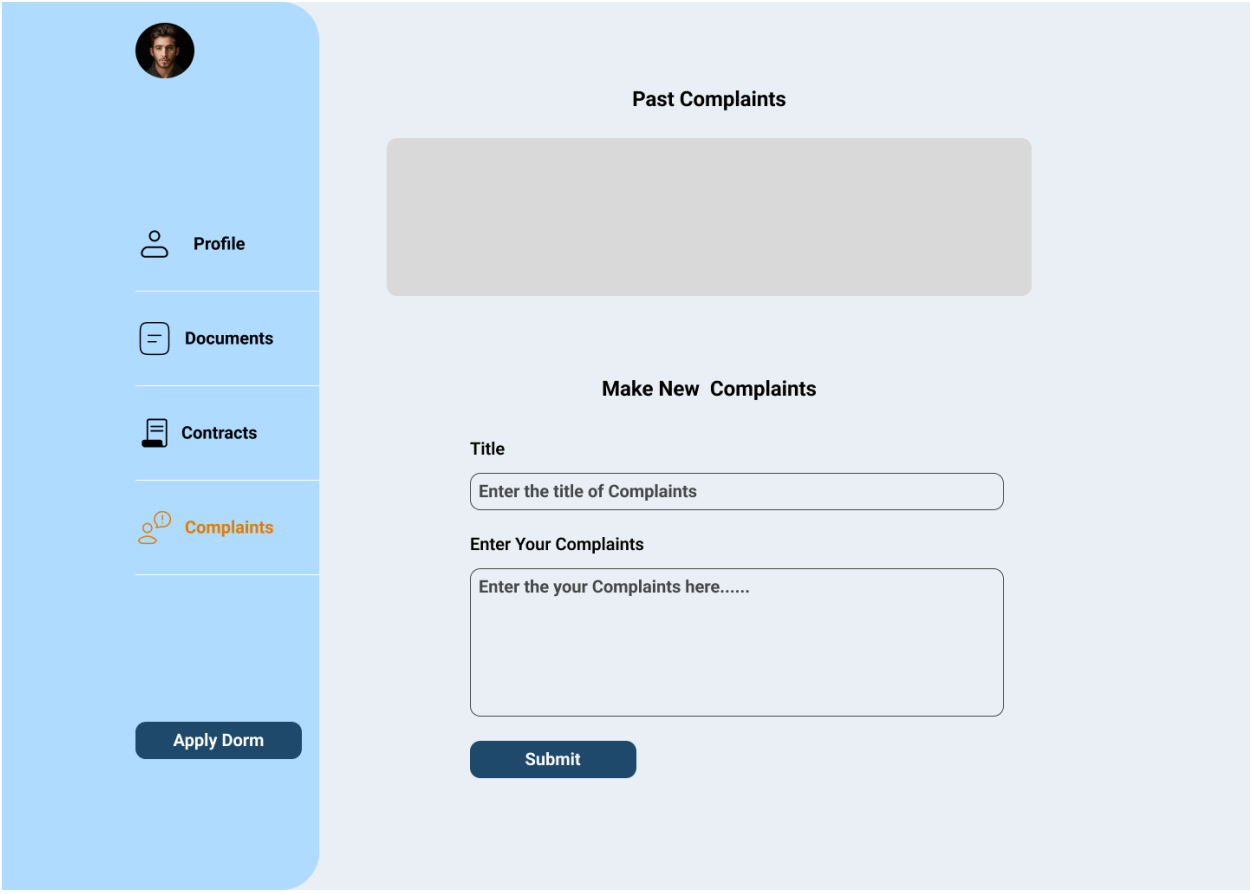


Figure 9: Mock Screen - Student Complaints

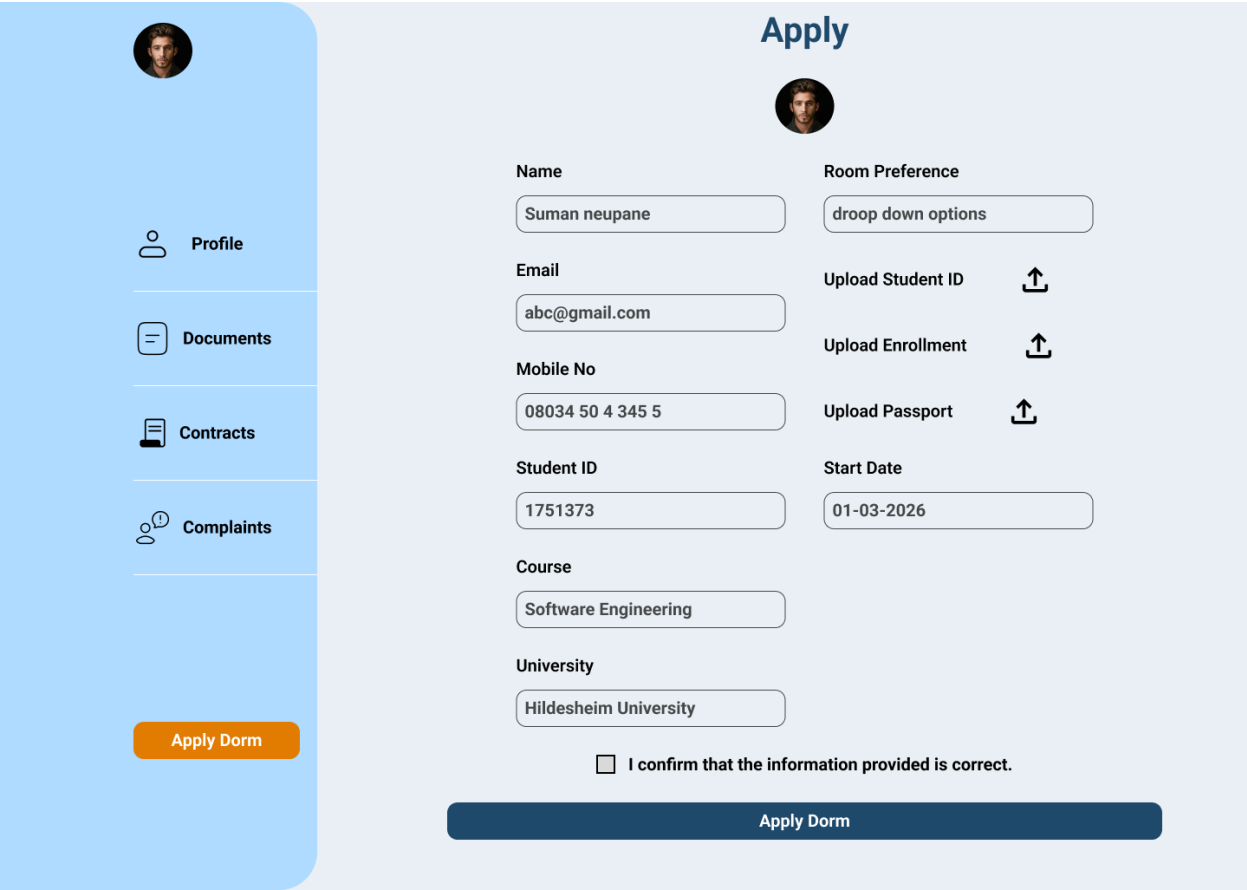


Figure 10: Mock Screen - Student Apply Dorm

Backend Design

The backend is implemented using Node.js with Express.js to provide RESTful APIs that support the Angular frontend. The system uses a MySQL relational database for structured data storage and Google Firebase Storage for managing uploaded files such as contracts and rent payment receipts. The backend also supports asynchronous processing for long-running tasks.

- **Key Features**

  - **Authentication and Authorization**
    - Secure user authentication using JWT.
    - Role-based access control for students and dormitory administrators.
  - **RESTful APIs**
    - **User Management**: Login, authentication, and role validation.
    - **Application Management**: Submit, review, accept, or reject dormitory applications.
    - **Contract Management**: Create, extend, and terminate dormitory contracts.
    - **Complaint Management**: Submit complaints and update their status.
    - **File Management**: Upload and download documents using Google Firebase Storage.
    - **Report Management**: Generate rent payment history reports.
  - **File Storage**
    - User-uploaded documents (contracts and payment receipts) are stored in Google Firebase Storage.
    - File metadata (file path, owner, upload date) is stored in the MySQL database.
    - Secure access rules ensure that files are accessible only to authorized users.
  - **Long-Running Backend Processing**
    - Rent payment report generation is handled as an asynchronous backend task.
    - The backend aggregates rent payment records from MySQL and generates report files.
    - Generated reports are stored in Firebase Storage and made available for download.
  - **Progress Tracking**
    - Backend services periodically update computation progress during report generation.
    - Progress information is exposed via APIs and consumed by the frontend to display visual progress indicators.

- **Database Schema Overview (MySQL)**

  - **Users**: Stores user credentials, roles, and authentication data.
  - **StudentProfiles**: Stores student-specific profile and academic information.
  - **Applications**: Tracks dormitory application data and decision status.
  - **Contracts**: Manages dormitory contract lifecycle information.
  - **RentPayments**: Stores structured rent payment records.
  - **Complaints**: Logs complaints and resolution status.
  - **Reports**: Stores report metadata and references to generated files in Firebase Storage.
  - **Files**: Stores file metadata such as storage paths, ownership, and upload timestamps.